

Preguntele al profe si podemos subir la misma
UNIVERSIDAD DE COSTA RICA

ESCUELA DE CIENCIAS DE LA COMPUTACIÓN E INFORMÁTICA
PROYECTO INTEGRADOR DE SISTEMAS OPERATIVOS Y REDES DE
COMUNICACIÓN

Proyecto del curso - Etapa II

Protocolo de comunicación y conexión TP (Trashing Protocol / SUPER SATÁN)

DOCENTES:

Prof. Maeva Murcia Meléndez

Prof. Francisco Arroyo

Estudiantes:

Grupo 2

I Semestre - 2026

Protocolo de comunicación y conexión - Proyecto: Etapa 2

I. Introducción

El presente protocolo define un conjunto de reglas para el intercambio de información entre componentes dentro de un sistema de consulta de figuras construidas con piezas de lego. El objetivo principal del protocolo es permitir que los clientes puedan:

- Consultar las figuras disponibles en el sistema.
- Obtener las piezas necesarias para construir una figura específica.

El protocolo sigue una arquitectura lógica de tipo cliente–intermediario–servidor, en la cual cada componente cumple un rol claramente definido y la comunicación se realiza mediante el intercambio de mensajes estructurados.

II. Componentes del sistema

1. *Cliente*

El cliente es la entidad que inicia la comunicación. Sus responsabilidades incluyen:

- Enviar solicitudes de consulta.
- Procesar y presentar las respuestas recibidas.

El cliente no interactúa directamente con los servidores de piezas. Todas las solicitudes son enviadas al intermediario, el cual actúa como punto único de acceso al sistema.

2. *Servidor intermedio (intermediario)*

El intermediario es el componente central del protocolo. Su responsabilidad principal es coordinar el flujo de comunicación entre el cliente y los servidores.

Cuando el intermediario recibe una solicitud, realiza las siguientes acciones:

- Interpreta el tipo de solicitud recibida.
- Determina si la solicitud requiere acceso a datos del sistema.
- Consulta su tabla de rutas para identificar el servidor adecuado.
- Reenvía la solicitud al servidor correspondiente.
- Procesa la respuesta recibida.
- Genera una respuesta final adecuada para el cliente.

3. *Servidor*

El servidor es responsable de almacenar y gestionar la información de las figuras. Ante una solicitud, el servidor:

- Verifica la existencia de la figura solicitada.
- Recupera la información correspondiente desde su almacenamiento.
- Construye una respuesta con los datos solicitados o con una indicación de error.

El servidor no interactúa directamente con el cliente, sino exclusivamente con el intermediario.

III. Formato de mensajes

Los mensajes en el protocolo se representan como estructuras compuestas por los siguientes campos:

- **tipo:** operación por realizar o respuesta generada.
- **figura:** nombre de la figura involucrada en la operación.
- **figuras:** lista de nombres de figuras
- **segmento:** mitad de la figura solicitada (1 o 2)
- **piezas:** listado de pares (pieza, cantidad) de la composición de una figura.

No todos los campos son utilizados en todos los mensajes; su uso depende del tipo de operación.

IV. Tipo de mensajes

El protocolo solicita un mínimo de acciones entre el intermediario con su servidor de piezas y los clientes, de esta manera se puede combinar con otros protocolos (btp, tinder Protocols, Chamuy protocol) que manejan la interacción de cliente con intermediario, el intermediario con cliente.

V. Respuestas

El protocolo define dos tipos principales de respuestas visibles para otro intermediario:

Respuesta de figura

El mensaje *INTERMEDIARY_RESPONSE* contiene:

- Nombre de la figura
- Segmento consultado
- Lista de piezas con sus cantidades

Respuesta de error

El mensaje *FIGURE_NOT_FOUND* indica que la figura solicitada no está disponible o que la solicitud no pudo ser procesada correctamente.

VI. Modelo de datos

Las figuras se componen de dos segmentos independientes, cada uno formado por un conjunto de piezas.

Cada pieza está definida por su nombre y cantidad.

VII. Bitácoras (Logs)

Las entidades del sistema generan registros asociados a los eventos de comunicación. Estos registros incluyen:

- Recepción de solicitudes
- Gestión y enrutamiento
- Generación de respuestas
- Ocurrencia de errores

El propósito de las bitácoras es proporcionar trazabilidad del sistema, facilitar el monitoreo de su comportamiento y apoyar procesos de depuración.

VIII. Tabla de Figuras del intermediario

El intermediario mantiene una estructura denominada **tabla de figuras** (o tabla de rutas), la cual permite asociar cada figura con uno o más servidores capaces de proveer su información.

El enrutamiento se realiza a través de sockets, pertenecientes a la clase VSocket. Cada conexión se gestiona mediante un hilo independiente, el cual se encarga de escuchar continuamente el canal de comunicación, (similar a un *listen()*), procesar solicitudes y enviar respuestas.

Además, el intermediario mantiene una tabla de **figuras globales**, que contiene todas las figuras conocidas a través de sus conexiones con otros intermediarios. Esta tabla permite validar rápidamente si una figura existe en la red antes de iniciar una búsqueda más costosa.

Durante la ejecución del sistema no se agregan nuevas figuras dinámicamente; por lo tanto, cualquier consulta siempre resultará en una figura válida o en un **FIGURE_NOT_FOUND**.

Uso:

"House"
"Orchid"
"Fish"
"Giraffe"

Cuando el intermediario recibe una solicitud de una figura específica:

1. Consulta la tabla de figuras globales.
2. Si la figura no se encuentra en la tabla de figuras, retorna **FIGURE_NOT_FOUND**.
3. De encontrarla, la busca en su propio servidor de figuras.
4. En caso de no estar en el servidor del intermediario, implementa un reenvío de la solicitud hacia el resto de intermediarios con el tipo de paquete correspondiente.
5. Si se obtiene la figura del otro intermediario se vuelve a comunicar en el canal.
6. Si no la encuentra procede a eliminar de su tabla de figuras globales y retorna:

FIGURE_NOT_FOUND

En caso de existir múltiples servidores asociados a una misma figura, el intermediario selecciona el primero en responder a la solicitud del fork.

IX. Comunicación entre intermediarios

Conexión a la red

Cada intermediario se conecta a la red de intermediarios a través de un switch centralizado (todos conocen el puerto y la ip de este switch). Tanto la conexión para el switch como para los intermediarios se maneja con el protocolo IPv4, sin encriptado SSL. Para crear la conexión:

- El intermediario envía una solicitud JOIN utilizando UDP.
- El switch realiza un reenvío (broadcast) hacia los demás intermediarios previamente registrados.
- Posteriormente, el switch almacena la dirección del nuevo intermediario.

Los demás intermediarios que reciben este JOIN, harán un HANDSHAKE con el intermediario que envió la solicitud, para establecer una conexión directa TCP/IP. Están establecidos unos puertos según sea necesaria la comunicación, pero para paquetes JOIN se utiliza el puerto 3030. Todos los intermediarios deben estar escuchando paquetes JOIN por UDP de otros intermediarios por el puerto 3030. El intermediario que reciba el paquete JOIN va a establecer una conexión TCP mediante la ip del remitente y puerto 3031, va a intercambiar paquetes HANDSHAKE y cerrará la conexión. En ese punto, ambos intermediarios conocerán sus tablas de figuras y sus direcciones ip para poder comunicarse posteriormente.

Intercambio de información

Cuando un intermediario recibe un HANDSHAKE de otro intermediario, estos intercambian sus tablas de figuras, permitiendo actualizar sus tablas de figuras. Los intermediarios también pueden enviar y recibir paquetes, de tipo:

- **INTERMEDIARY_REQUEST** (solicitudes de figuras)
- **INTERMEDIARY_RESPONSE** (respuestas con datos de figuras)

Con cada intercambio de paquetes HANDSHAKE que se haga, se asociarán las direcciones IP a las figuras en un contenedor lineal para consultar secuencialmente entre los intermediarios en el caso de consultar por alguna pieza de lego.

Manejo de conexiones

Por cada conexión entre intermediarios, se crea un hilo que maneje esa comunicación. De esta forma, se establece un contexto apropiado y distinto para cada comunicación. El uso de hilos es necesario, ya que distintos intermediarios no pueden comunicarse a través de un mismo canal individual. Cada conexión requiere su propio canal independiente para evitar interferencias en la comunicación.

El hilo se encarga tanto de la lectura como de la escritura en su canal correspondiente con el intermediario específico. Además, procesa los mensajes que recibe: si se trata de un `INTERMEDIARY_RESPONSE`, envía el paquete al cliente; o si es un `INTERMEDIARY_REQUEST`, procesa la solicitud según corresponda.

Cuando un intermediario recibe la respuesta de una solicitud hecha por un cliente específico, este puede reconocer a dicho cliente mediante la dirección que contiene el paquete enviado.

Cuando se intercambian paquetes `HANDSHAKE` entre intermediarios se cierra la conexión, pero se almacenan las direcciones entre ellos para luego poder comunicarse de nuevo.

Cuando un intermediario envía su paquete `JOIN` por UDP, otro intermediario lo recibirá y abrirá una conexión con el remitente mediante TCP y envía un paquete `HANDSHAKE`, luego el otro intermediario envía el suyo y cierra la conexión.

Definiciones de los paquetes

Nota: si el nombre de la figura tiene una coma se le pone un carácter“\” antes de la coma.

Paquete `INTERMEDIARY_JOIN`

- tipo `uint8_t` = `INTERMEDIARY_JOIN` : En el primer atributo se especifica que es un tipo de solicitud JOIN y busca conexión.
- `sourceIp_in_addr`: Dirección Ip del intermediario.

Paquete `INTERMEDIARY_HANDSHAKE`

- tipo `uint8_t` = `INTERMEDIARY_HANDSHAKE`
- `contentLength` `uint32_t`
- `content` `char[n]` =(nombre de cada figura en lista partida por comas)

Ejemplo de contenido: “**lion,giraffe,shark**”

Paquete `INTERMEDIARY_REQUEST`

- tipo `uint8_t` = `INTERMEDIARY_REQUEST`
- `mid` `uint8_t` : 1 primera, 2 segunda, 3 todas
- `contentLength` `uint8_t`
- `content` `char[n]` : nombre de la figura que se solicita

Paquete INTERMEDIARY_RESPONSE

- tipo uint8_t = **INTERMEDIARY_RESPONSE** : especifica que es un paquete de respuesta.
- mitad uint8_t : 1 primera, 2 segunda, 3 todas
- figureNameLength uint8_t : tamaño del nombre de la figura
- figureName char[n] : nombre de la figure
- contentLength uint32_t : tamaño del contenido
- content char[n] : (lista de piezas con cantidades de la figura solicitada)

El atributo content tiene la siguiente estructura:

[cantidad_pieza,nombre_pieza][cantidad_pieza,nombre_pieza]...

Paquete FIGURE_NOT_FOUND

- tipo uint8_t = **FIGURE_NOT_FOUND**
- contentLength uint8_t
- content char[n] : nombre de la figura que se solicitó

X. Manejo de errores

El protocolo contempla distintos escenarios de error y define comportamientos específicos para cada uno.

1. *Figura no encontrada*

Si la figura solicitada no existe en la tabla de rutas o no puede ser obtenida desde el servidor, el sistema responde con un paquete *FIGURE_NOT_FOUND*.

2. *Parámetros inválidos*

Si el cliente proporciona un segmento inválido, el sistema utiliza un valor por defecto (segmento 1) para garantizar la continuidad de la operación.

3. *Inconsistencias en la tabla de rutas*

Si una figura está registrada en la tabla de rutas pero los servidores asociados no pueden responder, el intermediario puede intentar con otros servidores disponibles.

Si ninguna alternativa es válida, se retorna *FIGURE_NOT_FOUND*.

XI. Valores en tipo

ID	Significado	Descripción	Cuando pasa
0	JOIN	Descubrimiento de forks en red	Cuando el fork se conecta a la red, lo envía a la red para descubrir forks.
1	HANDSHAKE	Confirmación de forks en red	Confirma al fork que mandó un join para intercambiar figuras.
2	INTERMEDIARY_REQUEST	Solicitud entre forks	Cuando un fork solicita una figura a otro fork.
3	INTERMEDIARY_RESPONSE	Respuesta entre forks	Cuando un fork responde a otro fork que le mandó un request.
4	FIGURE_NOT_FOUND	Respuesta de que no se encontró la figura que pidió el request	Cuando un fork responde a otro fork, y no encontró lo que le pidió.

XI. Limitaciones del protocolo

No existe una restricción además de los requisitos básicos entre cliente intermediario el intermediario servidor, pero la comunicación intermediario intermediario es muy estricta solo pueden pasarse las listas en un handshake y solo pueden solicitar una figura en un INTERMEDIARY_REQUEST

XII. Mecanismos de puertos

El intermediario debe escuchar paquetes JOIN por UDP de otros forks mediante el puerto 3030. Luego, debe escuchar paquetes INTERMEDIARY_REQUEST o HANDSHAKE de otros intermediarios mediante el puerto 3031.

El intermediario en su ciclo de escuchar por paquetes JOIN debe establecer una conexión directa TCP con el otro intermediario mediante el puerto 3031, enviarse mutuamente un paquete HANDSHAKE y cerrar su conexión. Este mecanismo es para que se intercambien las figuras que conocen solamente.

En el caso de paquetes malformados en solicitudes de conexión entre intermediarios, se deberá descartar la solicitud de conexión.

Cuando un intermediario quiere enviarle un paquete INTERMEDIARY_REQUEST a otro, abre una conexión TCP al puerto 3031 con la IP del otro intermediario.